

Friedrich-Alexander-Universität
Erlangen-Nürnberg (FAU)

NHR@FAU

Assignment 9: Code optimization, performance counters, barriers

Programming Techniques for Supercomputers (PTfS)

Muhammad Omar
Student IdM: to37gulo

Gehan Younes
Student IdM: qi39dyry

Ahmed Mohamed
Student IdM: gi10dafy

July 14, 2026

1 Optimizing Loop Code (25 Credits)

Consider the following loop:

```
int N = 16384;
double mat[N][N], s[N][N];

srand(1);

for (i = 0; i < N; ++i) {
    val = (double)rand();

    for (j = 0; j < N; ++j) {
        mat[j][i] = s[j][i] * sqrt(val);
    }
}
```

(a) Performance problems and optimizations

The main performance problems are:

1. **Bad memory access order:**

   C stores two-dimensional arrays in row-major order, so the last index should vary in the inner loop. However, the code accesses

   $$\texttt{mat[0][i]},\ \texttt{mat[1][i]},\ \texttt{mat[2][i]},\ldots$$

   with a stride of

   $$N \cdot 8 = 16384 \cdot 8 = 128 \text{ KiB}.$$

   This causes poor cache utilization, many TLB misses, inefficient prefetching, and prevents efficient SIMD vectorization.

2. **Repeated square-root calculation:**

   For a fixed value of i, `val` is constant, but `sqrt(val)` is calculated N times. It should be calculated only once for every i.

3. **Very large local arrays:**

   Each matrix occupies

   $$N^2 \cdot 8 = 2 \text{ GiB}.$$

   Therefore, both matrices require approximately 4 GiB. If they are local variables, they should be allocated dynamically instead of on the stack.

The loops cannot simply be interchanged because this would change the order of the calls to `rand()`. Instead, the random factors should first be calculated in the original order:

```
double factor[N];

srand(1);

for (int i = 0; i < N; ++i) {
```

```
    factor[i] = sqrt((double)rand());
}

#pragma omp parallel for schedule(static)
for (int j = 0; j < N; ++j) {
    #pragma omp simd
    for (int i = 0; i < N; ++i) {
        mat[j][i] = s[j][i] * factor[i];
    }
}
```

This version preserves the numerical results and provides:

- contiguous access to `mat` and `s`,
- only N square-root operations instead of N^2,
- SIMD vectorization of the inner loop,
- OpenMP parallelization over complete matrix rows.

Since `mat` is completely overwritten, non-temporal stores may also be used if `mat` is not read again immediately. This avoids the write-allocate traffic.

A compiler can normally vectorize the optimized inner loop. It may also move `sqrt(val)` outside the inner loop if floating-point exception and `errno` semantics allow it. However, it will generally not introduce the temporary factor array, preserve the `rand()` sequence while interchanging the loops, or add OpenMP parallelization automatically.

(b) Roofline model

The Roofline model is

$$P = \min\left(P_{\max}, I b_S\right) = \min\left(P_{\max}, \frac{b_S}{B_C}\right),$$

where $P_{\max}$ is the applicable in-core performance limit, b_S is the saturated memory bandwidth, and B_C is the code balance.

After optimization, the main matrix loop is

```
mat[j][i] = s[j][i] * factor[i];
```

The array `factor` has a size of only

$$16384 \cdot 8 = 128 \text{ KiB},$$

so it can remain in cache. Thus, for the main-memory Roofline model, one matrix update consists of one load from `s`, one store to `mat`, and one floating-point multiplication.

The N square-root operations are performed only once during the factor-generation step. Their amortized cost per matrix update is

$$\frac{3N}{N^2} = \frac{3}{N} = \frac{3}{16384} \approx 1.83 \times 10^{-4} \text{ cycles/update},$$

so their contribution is negligible compared with the N^2 matrix updates.

In-core performance limit With AVX-512, one 512-bit vector contains eight double-precision values. Although the core can execute more than one vector multiplication per cycle, the loop performs one store per update. The single 64-byte-per-cycle store path therefore limits the throughput to

$$P_{\max}^{\text{core}} = \frac{64 \text{ B/cycle}}{8 \text{ B/update}} = 8 \text{ updates/cycle/core}.$$

For one Fritz socket with 36 cores at 2.4 GHz,

$$P_{\max} = 36 \cdot 2.4 \times 10^9 \cdot 8 = 691.2 \times 10^9 \text{ updates/s}.$$

Since one update performs one floating-point multiplication,

$$\boxed{P_{\max} = 691.2 \text{ GFlop/s}}.$$

Memory bandwidth A per-socket bandwidth of approximately

$$b_S \approx 160 \text{ GB/s}$$

is used for the Roofline estimate. This is consistent with the measurements in Section 2, where one 18-core ccNUMA domain reached approximately

$$76.5 \text{ GB/s}.$$

Since one Fritz socket contains two such ccNUMA domains,

$$2 \cdot 76.5 \approx 153 \text{ GB/s},$$

which is close to the rounded value of 160 GB/s used in the model.

Ordinary write-allocate stores With ordinary cached stores, one update causes

$$\underbrace{8}_{\text{load } s} + \underbrace{8}_{\text{write allocate of } mat} + \underbrace{8}_{\text{writeback of } mat} = 24 \text{ B/update}.$$

Hence,

$$B_C = 24 \text{ B/update}.$$

The memory-bound Roofline limit is therefore

$$P_{\text{mem}} = \frac{160 \times 10^9}{24} \approx 6.67 \times 10^9 \text{ updates/s}.$$

Since there is one floating-point multiplication per update,

$$\boxed{P_{\text{mem}} \approx 6.67 \text{ GFlop/s}}.$$

The complete Roofline prediction is therefore

$$P = \min(691.2,\ 6.67) \text{ GFlop/s},$$

and thus

$$\boxed{P \approx 6.67 \text{ GFlop/s}}.$$

Therefore, the optimized loop with ordinary stores is clearly **memory-bandwidth bound**.

Improvement using non-temporal stores Since `mat` is completely overwritten, non-temporal stores can avoid the write-allocate transfer. The memory traffic then becomes

$$\underbrace{8}_{\text{load } s} + \underbrace{8}_{\text{store } mat} = 16 \text{ B/update.}$$

Thus,

$$P_{\text{mem}}^{\text{NT}} = \frac{160 \times 10^9}{16} = 10 \times 10^9 \text{ updates/s,}$$

or

$$\boxed{P_{\text{mem}}^{\text{NT}} = 10 \text{ GFlop/s.}}$$

The corresponding Roofline prediction is

$$P^{\text{NT}} = \min(691.2,\, 10) = 10 \text{ GFlop/s.}$$

Hence, non-temporal stores improve the predicted performance from approximately

$$6.67 \text{ GFlop/s}$$

to

$$10 \text{ GFlop/s,}$$

but the loop remains strongly limited by main-memory bandwidth.

2 Hardware Performance Counters (40 Credits)

The STREAM Triad kernel is

$$a[i] = b[i] + s \cdot c[i].$$

The objective of this experiment was to measure the data transfers between the processor cores, the cache hierarchy, and main memory using LIKWID hardware performance counters.

Experimental setup

The experiment was submitted from the Fritz login node `fritz1`. An interactive job with access to the hardware performance counters was allocated using

```
srun --nodes=1 \
     --ntasks=1 \
     --cpus-per-task=18 \
     --constraint=hwperf \
     --time=00:30:00 \
     --pty bash
```

The experimental configuration was:

Parameter	Value
Submission node	`fritz1`
Compute node	`f1088`
Processor	Intel Xeon Platinum 8360Y
Microarchitecture	Intel Ice Lake SP
Nominal clock frequency	2.40 GHz
Frequency setting	Cluster default/Turbo
ccNUMA domain	NUMA node 0
Physical CPUs in the domain	0–17
Cores used	1 and 18
Loop length	$N = 10^8$
Repetitions	$R = 10$
Total loop updates	$NR = 10^9$
Scalar value	$s = 3.0$
Compiler module	`intel/2025.2.0`
Compiler	Intel `icx`
LIKWID module	`likwid/5.5.1`

The NUMA topology was checked using

```
numactl --hardware
```

which showed four ccNUMA domains with 18 cores each. NUMA node 0 contains CPUs 0–17.

LIKWID logical cpuset notation was used for pinning:

```
export CPU1="L:N:0"
export CPU18="L:N:0-17"
export OMP_DYNAMIC=false
```

The modules were loaded using

```
module purge
module load intel
module load likwid
```

The code was compiled using

```
icx -O3 -xHost -qopenmp \
    -qopt-streaming-stores=never \
    -DLIKWID_PERFMON \
    -I/apps/likwid/5.5.1/include \
    stream_triad.c \
    -L/apps/likwid/5.5.1/lib \
    -Wl,-rpath,/apps/likwid/5.5.1/lib \
    -llikwid -pthread \
    -o stream_triad
```

The option `-qopt-streaming-stores=never` was used to prevent the compiler from generating non-temporal stores automatically. This is important because the experiment is intended to observe the behavior of ordinary cached stores and their write-allocate traffic.

(a) OpenMP parallelization and LIKWID instrumentation

The benchmark was adapted from the previous OpenMP accumulate benchmark. The repetition loop was placed inside one OpenMP parallel region, with an `omp for` work-sharing loop for every repetition.

```
#define _GNU_SOURCE

#include <stdio.h>
#include <stdlib.h>
#include <omp.h>
#include <likwid-marker.h>

int main(int argc, char **argv)
{
    const long N =
        (argc > 1) ? atol(argv[1]) : 100000000L;

    const long repetitions =
        (argc > 2) ? atol(argv[2]) : 10L;

    const double scalar = 3.0;

    double *a = NULL;
    double *b = NULL;
    double *c = NULL;

    if (posix_memalign((void **)&a, 64,
                       (size_t)N * sizeof(double)) != 0 ||
        posix_memalign((void **)&b, 64,
                       (size_t)N * sizeof(double)) != 0 ||
        posix_memalign((void **)&c, 64,
                       (size_t)N * sizeof(double)) != 0) {
        fprintf(stderr, "Memory allocation failed\n");
        free(a);
        free(b);
        free(c);
        return EXIT_FAILURE;
    }

    LIKWID_MARKER_INIT;

    #pragma omp parallel
    {
        LIKWID_MARKER_THREADINIT;
        LIKWID_MARKER_REGISTER("triad");

        /* Parallel first-touch initialization */
        #pragma omp for schedule(static)
        for (long i = 0; i < N; ++i) {
            a[i] = 0.0;
            b[i] = 1.0;
            c[i] = 2.0;
        }

        LIKWID_MARKER_START("triad");

        for (long r = 0; r < repetitions; ++r) {
            #pragma omp for schedule(static)
            for (long i = 0; i < N; ++i) {
                a[i] = b[i] + scalar * c[i];
            }
        }
```

```
        LIKWID_MARKER_STOP("triad");
    }

    LIKWID_MARKER_CLOSE;

    /* Prevent dead-code elimination */
    double checksum = 0.0;

    #pragma omp parallel for schedule(static) reduction(+:checksum)
    for (long i = 0; i < N; ++i) {
        checksum += a[i];
    }

    printf("N             = %ld\n", N);
    printf("Repetitions   = %ld\n", repetitions);
    printf("Threads       = %d\n", omp_get_max_threads());
    printf("Checksum      = %.2f\n", checksum);

    free(a);
    free(b);
    free(c);

    return EXIT_SUCCESS;
}
```

Listing 1: OpenMP STREAM Triad benchmark with LIKWID markers

The arrays were initialized in parallel before starting the LIKWID region. This provides first-touch NUMA placement and ensures that initialization traffic is excluded from the measured region.

The marker calls were placed outside the repetition loop:

```
LIKWID_MARKER_START("triad");

for (long r = 0; r < repetitions; ++r) {
    /* STREAM Triad loop */
}

LIKWID_MARKER_STOP("triad");
```

This starts and stops the hardware counters only once per thread. Placing the marker calls inside the repetition loop would repeatedly read and reset the performance counters, introducing unnecessary measurement overhead. Measuring all repetitions as one region also produces a longer and more reliable measurement. The reported LIKWID marker call count was one.

The checksum was

$$\sum_{i=0}^{N-1} a[i] = 10^8 \, (1 + 3 \cdot 2) = 7 \times 10^8,$$

which agrees with the measured value `700000000.00`.

(b) Main-memory traffic

The one-thread measurement was performed using

```
export OMP_NUM_THREADS=1

likwid-perfctr \
    -C "$CPU1" \
```

```
    -g MEM \
    -m \
    ./stream_triad 100000000 10 \
    | tee mem_1thread.txt
```

The 18-thread measurement was performed using

```
export OMP_NUM_THREADS=18

likwid-perfctr \
    -C "$CPU18" \
    -g MEM \
    -m \
    ./stream_triad 100000000 10 \
    | tee mem_18threads.txt
```

The total number of measured loop iterations was

$$N_{\text{updates}} = NR = 10^8 \times 10 = 10^9.$$

The traffic per loop iteration was calculated as

$$B_{\text{level}} = \frac{V_{\text{level}}}{NR}.$$

Since LIKWID reports decimal GBytes and the experiment performs exactly 10^9 updates, a measured value of x GBytes corresponds numerically to

$$x \text{ bytes/iteration.}$$

Expected memory traffic

For ordinary cached stores, one STREAM Triad iteration requires:

$$\underbrace{8}_{b[i]\text{ load}} + \underbrace{8}_{c[i]\text{ load}} + \underbrace{8}_{a[i]\text{ write allocate}} + \underbrace{8}_{a[i]\text{ writeback}} = 32 \text{ B/iteration.}$$

The expected split is therefore

$$B_{\text{read}} = 24 \text{ B/iteration}, \qquad B_{\text{write}} = 8 \text{ B/iteration.}$$

Measured results

Threads	Runtime [s]	Read [GB]	Write [GB]	Total [GB]	Bandwidth [GB/s]
1	1.6079	23.9967	8.0166	32.0134	19.9099
18	0.3178	16.2756	8.0375	24.3131	76.5481

Equivalently, in bytes per loop iteration:

Threads	Read [B/it.]	Write [B/it.]	Total [B/it.]
1	23.9967	8.0166	32.0134
18	16.2756	8.0375	24.3131

For one thread, the measured total traffic was

$$32.0134 \text{ B/iteration},$$

which differs from the expected 32 B/iteration by only about 0.04%. Thus, the measured volume agrees very well with the expected two loads, one write-allocate transfer, and one write-back.

For 18 threads, the measured traffic decreased to

$$24.3131 \text{ B/iteration}.$$

The write traffic remained close to 8 B/iteration, but the read traffic decreased from approximately 24 to 16.28 B/iteration.

The reason is the Ice Lake *SpecI2M* write-allocate evasion mechanism. Under high memory load, the processor recognizes the full cache-line output stream and avoids most of the read-for-ownership traffic for the output array a. Consequently, the old contents of a do not need to be read from main memory before the cache line is overwritten.

With complete write-allocate evasion, the memory traffic is

$$\underbrace{8}_{b[i]\text{ load}} + \underbrace{8}_{c[i]\text{ load}} + \underbrace{8}_{a[i]\text{ writeback}} = 24 \text{ B/iteration}.$$

The measured value of 24.3131 B/iteration is only about 1.3% above this limit, showing that most, but not all, write-allocate transfers were avoided.

The fact that nearly all cores displayed zero memory event counts is expected. The memory-controller events are shared uncore events. One hardware thread obtains the counter lock and reports the traffic for the complete measured domain. Therefore, the nonzero value, or the LIKWID `STAT Sum`, must be used instead of summing repeated per-core values.

(c) L2, L3, and main-memory traffic

The L3 measurement was performed using

```
export OMP_NUM_THREADS=18

likwid-perfctr \
    -C "$CPU18" \
    -g L3 \
    -m \
    ./stream_triad 100000000 10 \
    | tee l3_18threads.txt
```

The L2 measurement was performed using

```
likwid-perfctr \
    -C "$CPU18" \
    -g L2 \
    -m \
    ./stream_triad 100000000 10 \
    | tee l2_18threads.txt
```

The summed traffic results were:

Data path	Load/read [B/it.]	Evict/write [B/it.]	Total [B/it.]
L1–L2	24.0032	7.9894	31.9968
L2–lower hierarchy, LIKWID L3 group	24.0340	23.4613	47.4950
Main memory	16.2756	8.0375	24.3131

The L2 traffic is almost exactly

$$32 \text{ B/iteration}.$$

At the L1–L2 interface, approximately 24 B/iteration are loaded for the two input streams and the output-line allocation, while approximately 8 B/iteration are evicted for the modified output stream.

The main-memory traffic is lower because SpecI2M avoids most of the output write-allocate reads from DRAM. However, the corresponding cache-line allocation still exists inside the cache hierarchy, so the L2 traffic remains close to 32 B/iteration.

The L3 result is unusual:

$$B_{\mathrm{L3}} = 47.4950 \text{ B/iteration},$$

which is larger than both the L2 and memory data volumes.

The special property of the Ice Lake L3 cache is that it is a **non-inclusive victim cache**. It is not a mandatory intermediate cache through which every memory transfer must pass.

In particular:

- Data loaded from memory may be placed directly in a private L2 cache without becoming resident in L3.
- Lines evicted from L2 may be inserted into L3 as victims.
- Clean L2 victims may either be placed in L3 or dropped.
- Dirty victims may eventually be written to main memory.

Consequently, the LIKWID L3 group does not represent only physical L3-cache hits. Its load event counts cache lines entering L2 from the lower hierarchy, which may come from either L3 or memory. Its eviction event counts lines leaving L2 toward L3 or memory.

For this STREAM Triad run, approximately

$$24.0340 \text{ B/iteration}$$

entered the private L2 caches, while approximately

$$23.4613 \text{ B/iteration}$$

left L2 toward the lower hierarchy. This outgoing traffic contains not only the dirty output stream but also many clean input lines that are transferred to the victim L3 cache. Only

$$0.5727 \text{ B/iteration}$$

were reported as silently dropped cache lines. Therefore, L3 traffic does not have to equal the memory traffic, and it does not have to lie between the L2 and main-memory data volumes. This behavior is a direct consequence of the non-inclusive victim-cache organization of the Ice Lake L3 cache.

3 Barrier Pain (35 Credits)

Barrier synchronization forces all threads in an OpenMP team to wait until every thread has reached the same synchronization point. The goal of this experiment was to quantify the cost of an explicit OpenMP barrier in processor cycles and to study how this cost changes with the number of participating threads.

(a) Barrier benchmark

The barrier cost was measured by repeatedly executing explicit OpenMP barriers inside one persistent parallel region. This excludes the cost of repeatedly creating and destroying an OpenMP thread team.

The benchmark performs:

- 1000 warm-up barriers to reduce first-use effects;
- 100000 measured barriers per trial;
- seven independent trials;
- reporting of the median, minimum, and maximum barrier costs.

Each thread independently measures the elapsed time of the complete barrier sequence using `omp_get_wtime()`. The maximum elapsed time among all threads is used because the complete barrier sequence finishes only when the slowest thread has finished.

Since the processor frequency was fixed at 2 GHz, the average barrier cost was calculated as

$$C_{\text{barrier}} = \frac{t_{\max} \cdot 2 \times 10^9}{N_{\text{barriers}}},$$

where $t_{\max}$ is the maximum elapsed time among all threads and $N_{\text{barriers}} = 100000$.

```
#include <stdio.h>
#include <stdlib.h>
#include <omp.h>

#define WARMUP_BARRIERS   1000
#define MEASURED_BARRIERS 100000
#define TRIALS            7
#define FREQUENCY_HZ      2.0e9

static int compare_double(const void *a, const void *b)
{
    const double x = *(const double *)a;
    const double y = *(const double *)b;

    return (x > y) - (x < y);
}

int main(void)
{
    double samples[TRIALS];
    double max_elapsed = 0.0;
    int nthreads = 1;

    omp_set_dynamic(0);

    #pragma omp parallel shared(samples, max_elapsed, nthreads)
    {
        #pragma omp single
        {
            nthreads = omp_get_num_threads();
        }

        /* Warm up the OpenMP runtime and barrier implementation. */
        for (int i = 0; i < WARMUP_BARRIERS; ++i) {
```

```
            #pragma omp barrier
        }

        for (int trial = 0; trial < TRIALS; ++trial) {

            #pragma omp single
            {
                max_elapsed = 0.0;
            }

            const double start = omp_get_wtime();

            for (long i = 0; i < MEASURED_BARRIERS; ++i) {
                #pragma omp barrier
            }

            const double elapsed = omp_get_wtime() - start;

            #pragma omp critical
            {
                if (elapsed > max_elapsed) {
                    max_elapsed = elapsed;
                }
            }

            #pragma omp barrier

            #pragma omp single
            {
                samples[trial] =
                    max_elapsed * FREQUENCY_HZ
                    / (double)MEASURED_BARRIERS;
            }
        }
    }

    qsort(samples, TRIALS, sizeof(double), compare_double);

    printf("%d %.3f %.3f %.3f\n",
           nthreads,
           samples[TRIALS / 2],
           samples[0],
           samples[TRIALS - 1]);

    return EXIT_SUCCESS;
}
```

Listing 2: OpenMP barrier benchmark

The output columns are

`threads median_cycles minimum_cycles maximum_cycles`.

Using many barrier repetitions makes the timer overhead negligible compared with the measured synchronization time. The warm-up phase reduces runtime initialization effects, while the minimum-to-maximum range provides an indication of measurement variability.

The results were checked for plausibility by verifying that the barrier cost generally increased with thread count. Small non-monotonic variations are expected because of operating-system noise, thread-arrival imbalance, and runtime-specific barrier algorithms.

(b) Barrier scaling on one Fritz node

Experimental setup

The experiment was submitted from `fritz1` and executed on compute node `f0810`. The full node, containing 72 physical cores, was allocated at a fixed frequency of 2 GHz using

```
srun --nodes=1 \
     --ntasks=1 \
     --cpus-per-task=72 \
     --cpu-freq=2000000-2000000:performance \
     --time=01:00:00 \
     --pty bash
```

The node contains four ccNUMA domains:

ccNUMA domain	CPU IDs
0	0–17
1	18–35
2	36–53
3	54–71

Thus, every ccNUMA domain contains 18 cores, and each socket contains two ccNUMA domains and 36 cores.

The same OpenMP placement settings were used for both compilers:

```
export OMP_DYNAMIC=false
export OMP_PLACES=cores
export OMP_PROC_BIND=close
```

This binds each OpenMP thread to one physical core and places threads on nearby cores before extending the team into another ccNUMA domain or socket.

Intel compiler

The default Intel compiler was Intel oneAPI 2025.2.0. The benchmark was compiled using

```
module load intel
icx -O3 -qopenmp barrier.c -o barrier_icx
```

The measurements for thread counts from 1 to 72 were generated using

```
echo "# threads median_cycles min_cycles max_cycles" \
    > intel_barrier.dat

for T in $(seq 1 72); do
    OMP_NUM_THREADS=$T ./barrier_icx \
        >> intel_barrier.dat
done
```

GNU compiler

The GNU benchmark was compiled with GCC 14.2.0:

```
module load gcc/14.2.0
gcc -O3 -fopenmp barrier.c -o barrier_gcc
```

The measurements were generated using

```
echo "# threads median_cycles min_cycles max_cycles" \
    > gcc_barrier.dat

for T in $(seq 1 72); do
    OMP_NUM_THREADS=$T ./barrier_gcc \
        >> gcc_barrier.dat
done
```

Results

Figure 1 shows the median barrier latency. The shaded regions represent the minimum-to-maximum range over the seven trials.

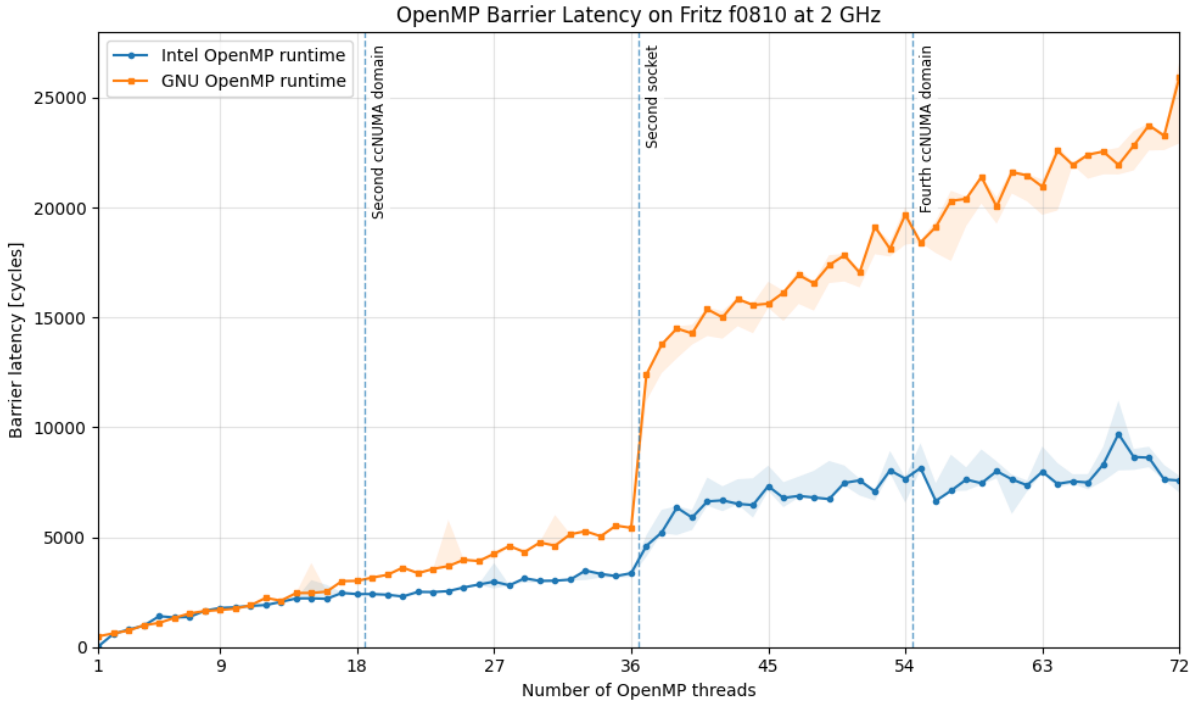


Figure 1: OpenMP barrier latency on Fritz compute node `f0810` at a fixed frequency of 2 GHz. The dashed vertical lines mark transitions to additional ccNUMA domains and to the second socket.

Selected median measurements are shown in Table 1.

Table 1: Selected median barrier latencies in processor cycles.

Threads	Intel OpenMP	GNU OpenMP
1	27.9	487.2
18	2410.9	3012.4
36	3363.3	5424.8
37	4607.1	12403.8
54	7662.5	19673.2
55	8148.9	18411.2
72	7580.4	25936.6

For both OpenMP runtimes, the barrier latency generally increases with the number of

participating threads. This is expected because more threads must communicate their arrival and wait for the slowest thread before execution can continue.

From 1 to 18 threads, all threads fit within one ccNUMA domain. Between 19 and 36 threads, the synchronization team extends into the second ccNUMA domain of the same socket. This causes a gradual increase in barrier cost, but communication remains within one processor socket.

The largest discontinuity occurs between 36 and 37 threads. At 37 threads, the OpenMP team starts using the second processor socket. The Intel barrier cost increases from

$$3363.3 \text{ cycles}$$

to

$$4607.1 \text{ cycles},$$

which is an increase of approximately 37%.

For the GNU runtime, the same transition increases the barrier cost from

$$5424.8 \text{ cycles}$$

to

$$12403.8 \text{ cycles},$$

which is more than a factor of two. This shows that cross-socket synchronization is substantially more expensive, especially for the GNU OpenMP runtime.

At 72 threads, the measured median latency was

$$7580.4 \text{ cycles}$$

for the Intel OpenMP runtime and

$$25936.6 \text{ cycles}$$

for the GNU OpenMP runtime. Thus, the GNU barrier required approximately

$$\frac{25936.6}{7580.4} \approx 3.42$$

times as many cycles as the Intel barrier at the full-node thread count.

The Intel OpenMP runtime therefore scales significantly better for this barrier benchmark. The difference is mainly caused by the barrier algorithms, waiting policies, and topology awareness implemented in the two OpenMP runtime libraries. The optimization level has little direct effect on the measured region because it contains synchronization operations rather than arithmetic work.

The curves are not strictly monotonic. For example, some higher thread counts have slightly lower median times than the preceding measurements. Such fluctuations are plausible because barrier latency depends on the arrival time of the slowest thread and can be affected by operating-system activity, runtime scheduling, and temporary thread imbalance. Nevertheless, the overall increase with thread count and the large cross-socket penalty confirm the plausibility of the measurements.